Visualizing w_is and f_is

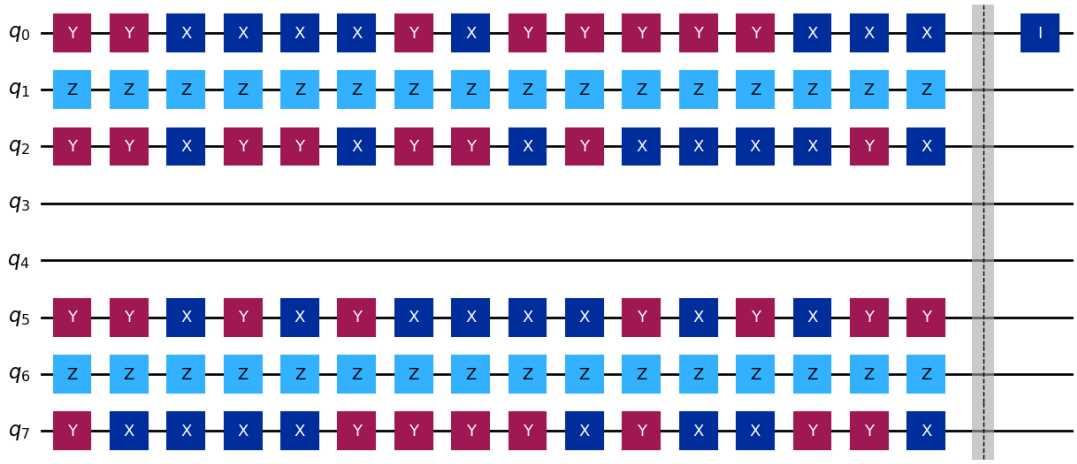
October 18, 2024

[3]:
```
from qiskit import QuantumCircuit

# Function to extract distinct Pauli strings from a file (ignores the coefficients)
def extract_unique_pauli_strings(file_path):
    """
    Function to extract distinct Pauli strings (ignoring coefficients) from a file.

    Args:
    file_path (str): Path to the file to be processed.

    Returns:
    A set of unique Pauli strings.
    """
    unique_pauli_strings = set()  # Use a set to store only distinct strings
    with open(file_path, 'r') as file:
        for line in file:
            # Split the line by '*' and take the Pauli string (ignoring coefficients)
            try:
                pauli_string = line.split('*')[1].strip()  # Ignore the coefficient and get only the Pauli string
                unique_pauli_strings.add(pauli_string)  # Add the string to the set
            except IndexError:
                continue  # Skip lines that do not contain valid Pauli strings
    return unique_pauli_strings

# Function to create a quantum circuit for each Pauli string
def add_pauli_to_circuit(qc, pauli_string):
    """
    Function to add Pauli operations (X, Y, Z) to a quantum circuit based on the Pauli string.

    Args:
    qc (QuantumCircuit): Quantum circuit to modify.
```

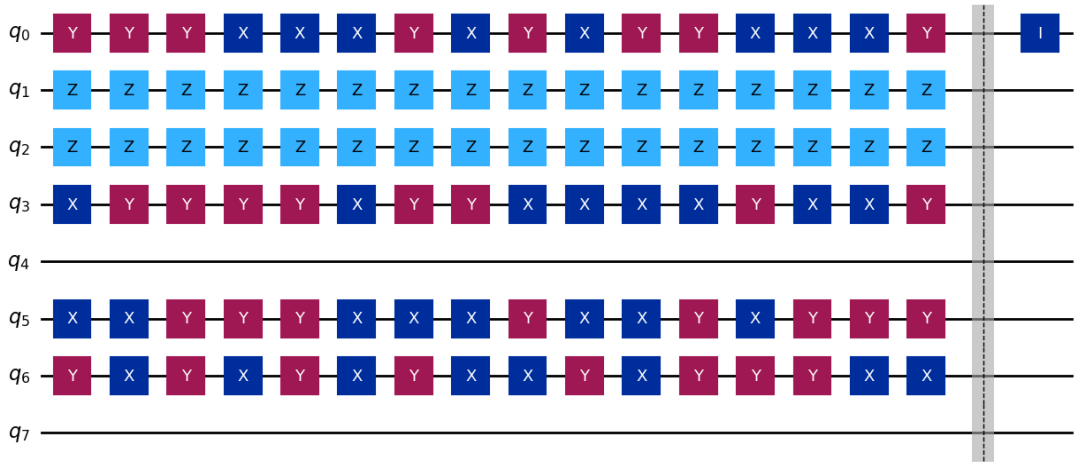
```
    pauli_string (str): The Pauli string to apply to the circuit.

    Returns:
    QuantumCircuit: The updated circuit.
    """
    for idx, pauli in enumerate(pauli_string):
        if pauli == 'X':
            qc.x(idx)
        elif pauli == 'Y':
            qc.y(idx)
        elif pauli == 'Z':
            qc.z(idx)
    return qc

# Paths to the f1 to f6 files
file_paths_f = {
    'f1': '/home/bruna/pauli_product_output_f1.txt',
    'f2': '/home/bruna/pauli_product_output_f2.txt',
    'f3': '/home/bruna/pauli_product_output_f3.txt',
    'f4': '/home/bruna/pauli_product_output_f4.txt',
    'f5': '/home/bruna/pauli_product_output_f5.txt',
    'f6': '/home/bruna/pauli_product_output_f6.txt'
}

# Loop through each file, extract distinct Pauli strings, and create separate␣
 ↪circuits
circuit_depths_f = {}

for file_label, file_path in file_paths_f.items():
    # Extract distinct Pauli strings
    pauli_strings = extract_unique_pauli_strings(file_path)

    # Create a quantum circuit for this file
    circuit = QuantumCircuit(8)

    for pauli_string in pauli_strings:
        circuit = add_pauli_to_circuit(circuit, pauli_string)

    # Add a barrier for clarity and insert an identity gate to mark the end of␣
 ↪the file's circuit
    circuit.barrier()
    circuit.id(0)  # Insert identity gate as a marker

    # Calculate and store the circuit depth
    circuit_depth = circuit.depth()
    circuit_depths_f[file_label] = circuit_depth
```

```
    # Draw the circuit (only displaying one circuit at a time)
    print(f"Circuit for {file_label}:")
    display(circuit.draw('mpl'))  # Use display to show each circuit
```

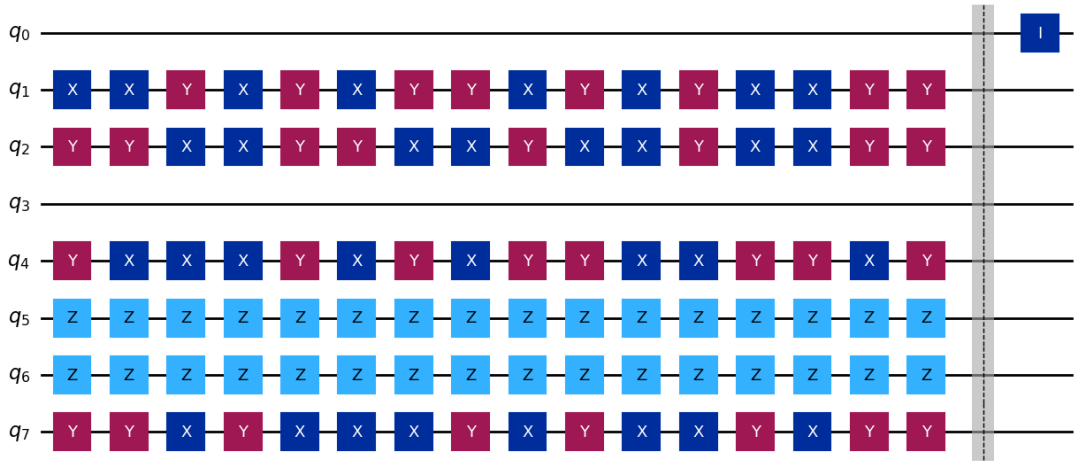
```
Circuit for f1:
```



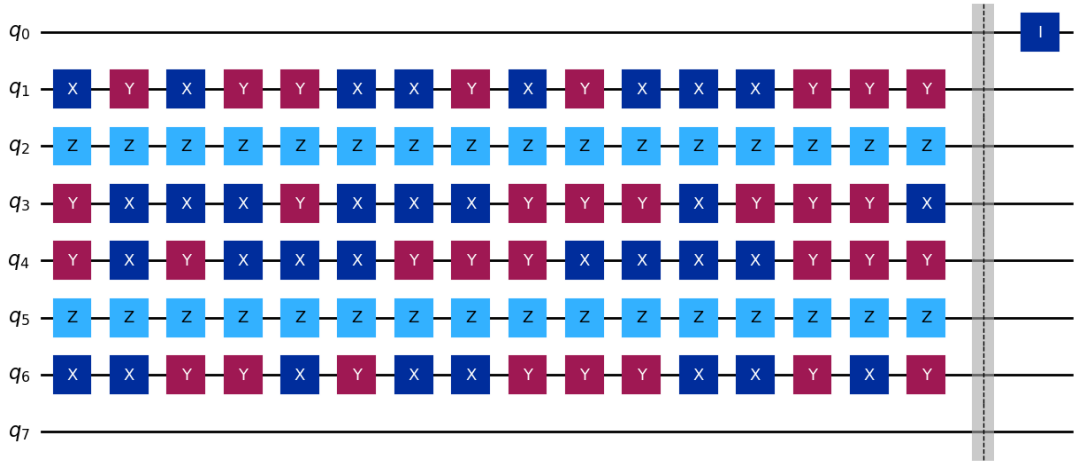

```
Circuit for f2:
```



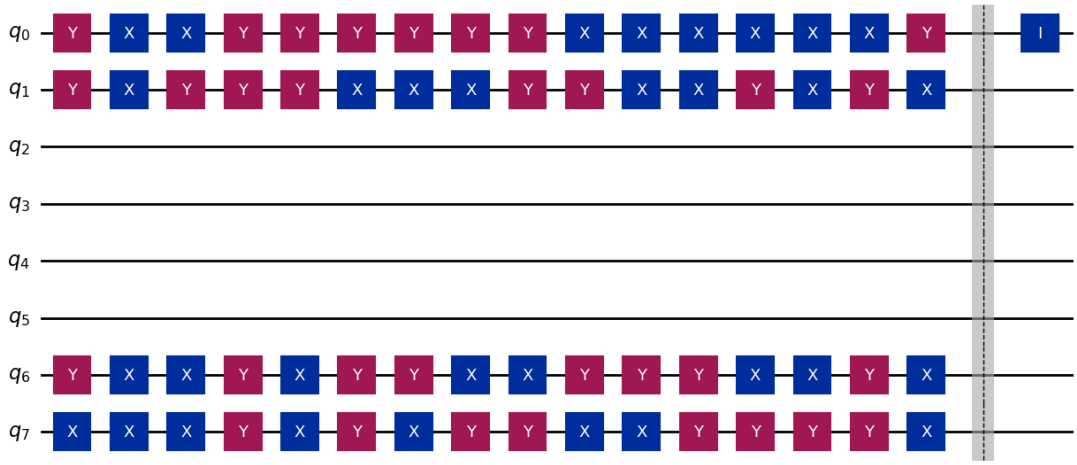

```
Circuit for f3:
```



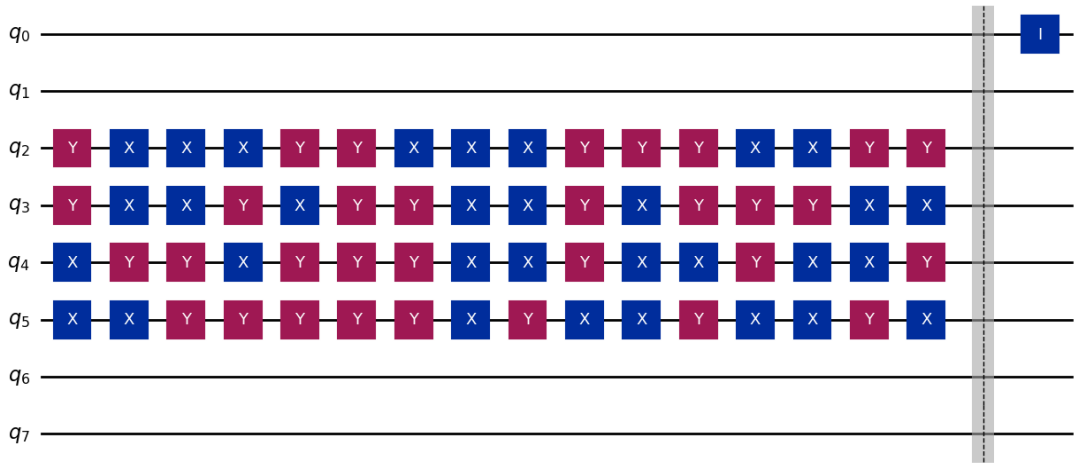
Circuit for f4:



Circuit for f5:



Circuit for f6:



[5]:
```
# Print the circuit depths
print("\nCircuit depths (complexity) for each f-file:")
for file_label, depth in circuit_depths_f.items():
    print(f"{file_label}: {depth}")
```

```
Circuit depths (complexity) for each f-file:
f1: 17
f2: 17
f3: 17
f4: 17
f5: 17
```

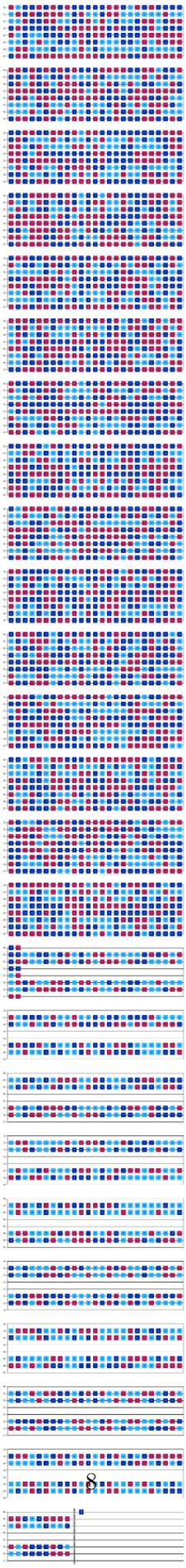
```
f6: 17
```

[7]:
```
# Calculating the NO JW mapping
```

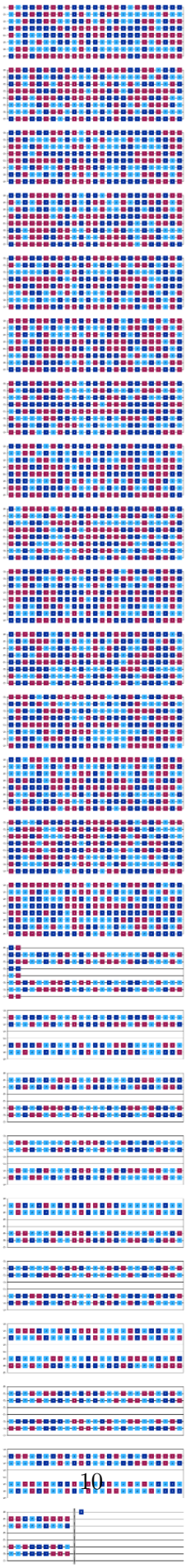
[11]:
```
from qiskit import QuantumCircuit

# Function to extract distinct Pauli strings from a file (ignores the␣
 ↪coefficients)
def extract_unique_pauli_strings(file_path):
    """
    Function to extract distinct Pauli strings (ignoring coefficients) from a␣
 ↪file.

    Args:
    file_path (str): Path to the file to be processed.

    Returns:
    A set of unique Pauli strings.
    """
    unique_pauli_strings = set()  # Use a set to store only distinct strings
    with open(file_path, 'r') as file:
        for line in file:
            # Split the line by '*' and take the Pauli string (ignoring␣
 ↪coefficients)
            try:
                pauli_string = line.split('*')[1].strip()  # Ignore the␣
 ↪coefficient and get only the Pauli string
                unique_pauli_strings.add(pauli_string)  # Add the string to the␣
 ↪set
            except IndexError:
                continue  # Skip lines that do not contain valid Pauli strings
    return unique_pauli_strings

# Function to create a quantum circuit for each Pauli string
def add_pauli_to_circuit(qc, pauli_string):
    """
    Function to add Pauli operations (X, Y, Z) to a quantum circuit based on␣
 ↪the Pauli string.

    Args:
    qc (QuantumCircuit): Quantum circuit to modify.
    pauli_string (str): The Pauli string to apply to the circuit.

    Returns:
    QuantumCircuit: The updated circuit.
    """
    for idx, pauli in enumerate(pauli_string):
```

```
        if pauli == 'X':
            qc.x(idx)
        elif pauli == 'Y':
            qc.y(idx)
        elif pauli == 'Z':
            qc.z(idx)
    return qc

# Paths to the w1 to w6 files
file_paths_w = {
    'w1': '/home/bruna/pauli_product_output_w1.txt',
    'w2': '/home/bruna/pauli_product_output_w2.txt',
    'w3': '/home/bruna/pauli_product_output_w3.txt',
    'w4': '/home/bruna/pauli_product_output_w4.txt',
    'w5': '/home/bruna/pauli_product_output_w5.txt',
    'w6': '/home/bruna/pauli_product_output_w6.txt'
}

# Loop through each file, extract distinct Pauli strings, and create separate␣
 ↪circuits
circuit_depths = {}

for file_label, file_path in file_paths_w.items():
    # Extract distinct Pauli strings
    pauli_strings = extract_unique_pauli_strings(file_path)

    # Create a quantum circuit for this file
    circuit = QuantumCircuit(8)

    for pauli_string in pauli_strings:
        circuit = add_pauli_to_circuit(circuit, pauli_string)

    # Add a barrier for clarity and insert an identity gate to mark the end of␣
 ↪the file's circuit
    circuit.barrier()
    circuit.id(0)  # Insert identity gate as a marker

    # Calculate and store the circuit depth
    circuit_depth = circuit.depth()
    circuit_depths[file_label] = circuit_depth

    # Draw the circuit (only displaying one circuit at a time)
    print(f"Circuit for {file_label}:")
    display(circuit.draw('mpl'))  # Use display to show each circuit
```

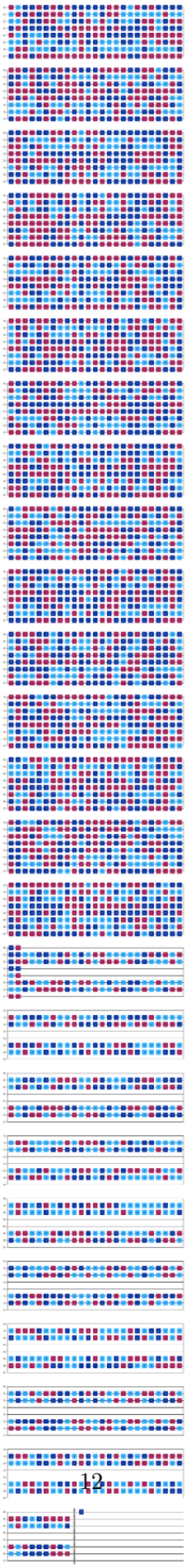
```
Circuit for w1:
```



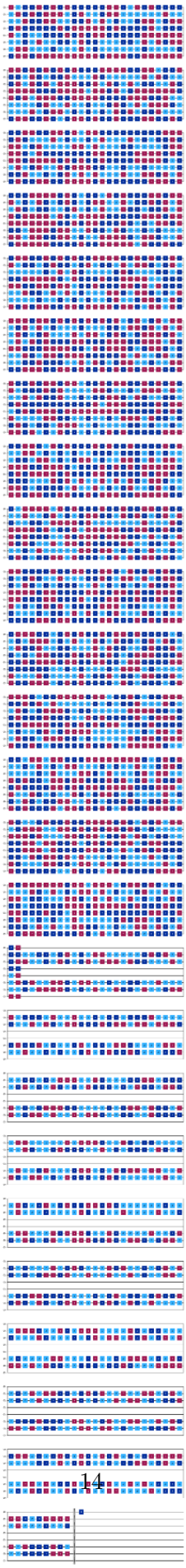
Circuit for w2:



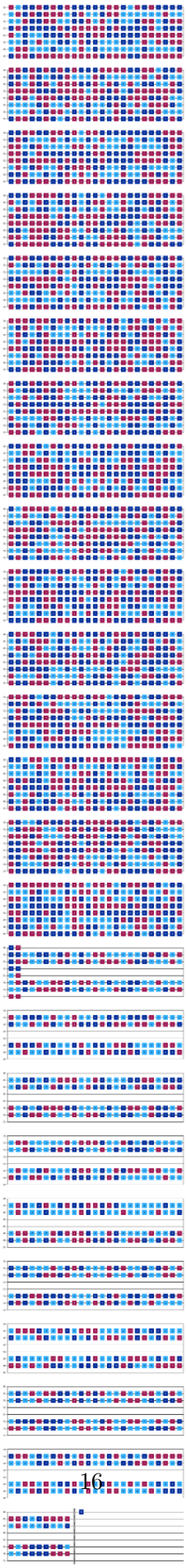
Circuit for w3:



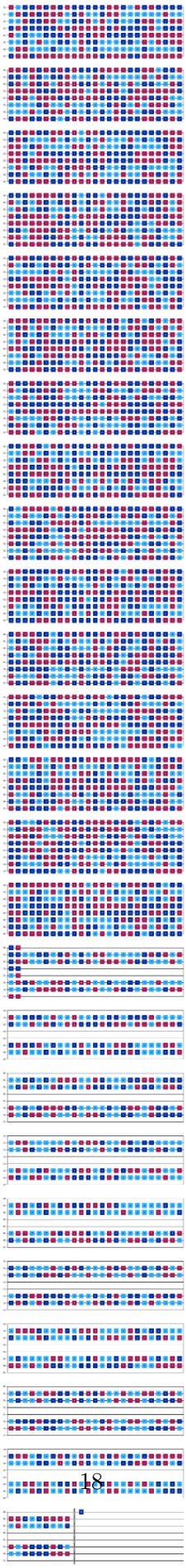
Circuit for w4:



Circuit for w5:



Circuit for w6:



[12]:
```
# Print the circuit depths
print("\nCircuit depths (complexity) for each w-file:")
for file_label, depth in circuit_depths.items():
    print(f"{file_label}: {depth}")
```

```
Circuit depths (complexity) for each w-file:
w1: 610
w2: 610
w3: 610
w4: 610
w5: 610
w6: 610
```

[ ]:

[ ]: